

Resumen Teórico: Desarrollo Backend con Node.js 2025

Autor: Basado en el curso de Backend Web 2025
Enfoque: Teoría conceptual con ejemplos prácticos
Versión: Node.js 24 (Active LTS - hasta abril 2028)

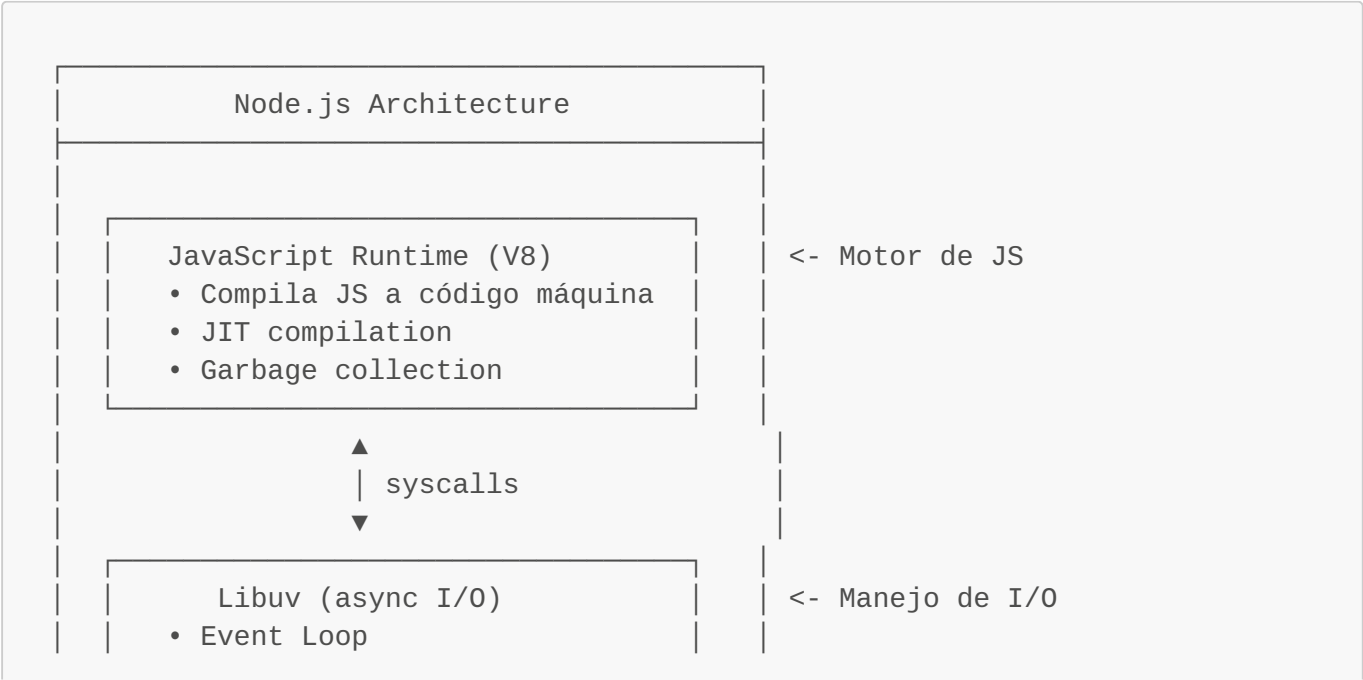
Tabla de Contenidos

- 1. Fundamentos Arquitectónicos
- 2. Asincronía y Event Loop
- 3. HTTP y Comunicación en Red
- 4. Express: Capa de Aplicación
- 5. Bases de Datos Relacionales vs NoSQL
- 6. Validación y Manejo de Errores
- 7. Autenticación y Autorización
- 8. Documentación y Testing
- 9. Bases de Datos SQL Modernas
- 10. Comunicación Bidireccional en Tiempo Real
- 11. Infraestructura y Deployment
- 12. Tipado Estático con TypeScript
- 13. Gestión de Archivos y Storage

1. Fundamentos Arquitectónicos

1.1 Arquitectura de Node.js

Node.js se basa en dos componentes fundamentales:



- Thread Pool (4-128 threads)
- File system, network, etc.

V8 (motor JavaScript):

- Desarrollado por Google para Chrome
- Compila JavaScript a código máquina nativo
- JIT (Just-In-Time) compilation para optimización
- Incluye garbage collector generacional

Libuv (async I/O library):

- Abstrae las diferencias del SO
- Implementa el Event Loop
- Maneja operaciones de I/O asíncronas
- Thread pool para operaciones que no pueden ser asíncronas

1.2 ECMAScript Modules (ESM)

Node.js moderno (v14+) soporta ESM como estándar:

```
// package.json
{
  "type": "module",
  "engines": {
    "node": ">=22.0.0"
  }
}

// Imports
import express from 'express';
import { readFile } from 'fs/promises';

// Top-level await (característica ESM)
const data = await readFile('file.txt', 'utf-8');
```

Ventajas ESM sobre CommonJS:

- Estándar de la industria (TC39)
- Tree-shaking automático
- Top-level await
- Mejor rendimiento

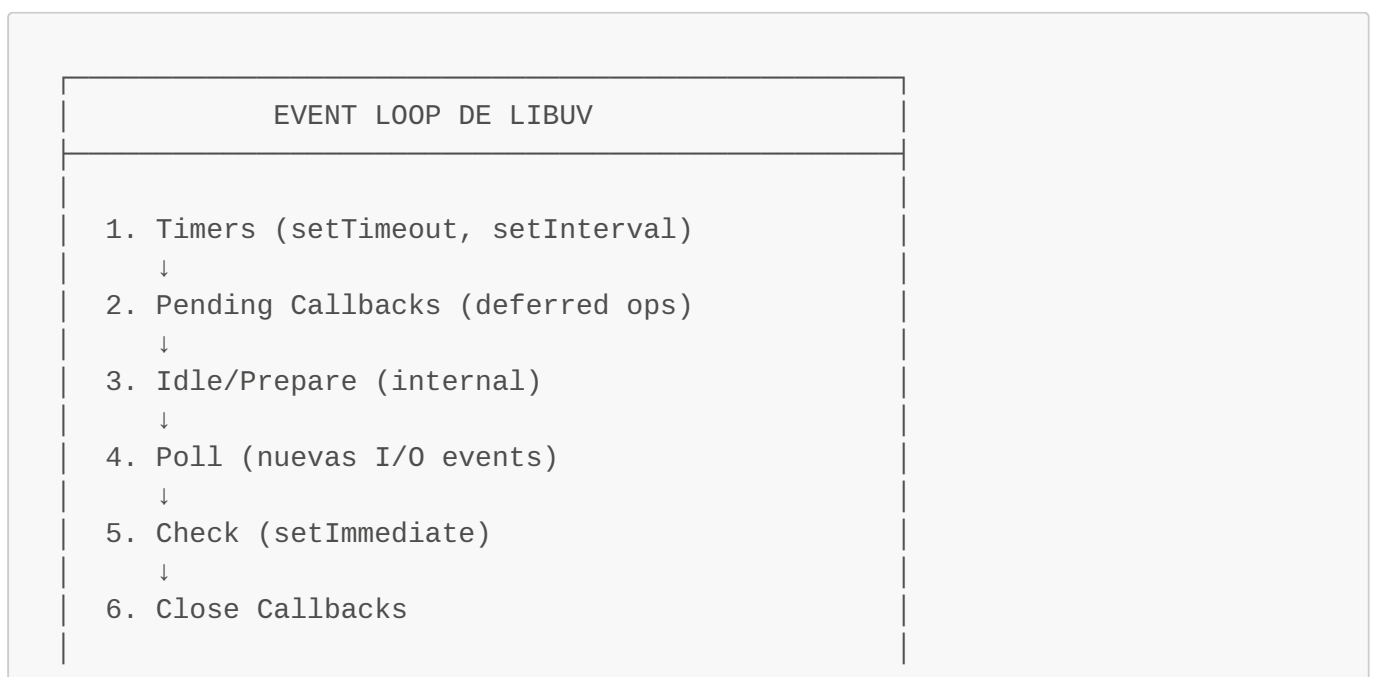
1.3 Modelo de Capas en Node.js



2. Asincronía y Event Loop

2.1 Modelo de Event Loop

El Event Loop es el corazón de Node.js:



Entre fases → Procesar Microtasks

- `Promise.then()`
- `queueMicrotask()`

Prioridades de ejecución:

1. **Call Stack** (código síncrono)
2. **`process.nextTick()`** (máxima prioridad asincrónica)
3. **Microtasks** (Promises, `queueMicrotask`)
4. **Macrotasks** (`setTimeout`, I/O, `setImmediate`)

2.2 Patrones de Asincronía

Callbacks (antiguo, evitar):

```
fs.readFile('file.txt', (err, data) => {  
  if (err) console.error(err);  
  else console.log(data);  
});
```

Promises (mejor):

```
fs.promises.readFile('file.txt')  
  .then(data => console.log(data))  
  .catch(err => console.error(err));
```

Async/Await (recomendado):

```
async function readAndProcess() {  
  try {  
    const data = await fs.promises.readFile('file.txt');  
    console.log(data);  
  } catch (err) {  
    console.error(err);  
  }  
}
```

2.3 Streams (Manejo Eficiente de Datos)

Los streams evitan cargar todo en memoria:

```
// Ineficiente: carga todo en memoria  
const data = fs.readFileSync('giant-file.txt');
```

```
// Eficiente: procesa en chunks
const stream = fs.createReadStream('giant-file.txt');
stream.on('data', chunk => {
  console.log(`Recibido ${chunk.length} bytes`);
});
```

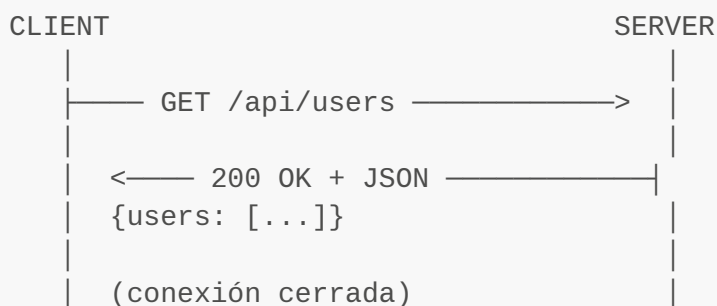
Conceptualmente:

- **Readable:** fuente de datos (archivo, red, etc.)
- **Writable:** destino de datos
- **Transform:** modifica datos mientras fluyen
- **Duplex:** tanto lectura como escritura

3. HTTP y Comunicación en Red

3.1 Protocolo HTTP

HTTP es stateless: cada petición es independiente



Métodos HTTP principales:

- **GET:** Obtener recurso (idempotente)
- **POST:** Crear recurso
- **PUT:** Reemplazar completamente (idempotente)
- **PATCH:** Actualizar parcialmente
- **DELETE:** Eliminar (idempotente)

Códigos de estado:

- **2xx:** Éxito (200 OK, 201 Created, 204 No Content)
- **3xx:** Redirección (301 Moved, 304 Not Modified)
- **4xx:** Error cliente (400 Bad Request, 401 Unauthorized, 404 Not Found)
- **5xx:** Error servidor (500 Internal Error, 503 Service Unavailable)

3.2 Arquitectura REST

REST (Representational State Transfer) define principios para APIs:

```
// Teoría:
// - Recursos identificables por URI
// - Métodos HTTP para operaciones
// - Representaciones en JSON
// - Stateless

// Ejemplo práctico:
GET    /api/users           // Listar todos
GET    /api/users/123       // Obtener uno
POST   /api/users           // Crear
PUT    /api/users/123       // Reemplazar
PATCH /api/users/123       // Actualizar
DELETE /api/users/123       // Eliminar

// Respuesta estándar
{
  success: true,
  data: { ... },
  timestamp: "2025-01-15T10:30:00Z"
}
```

4. Express: Capa de Aplicación

4.1 Concepto de Middleware

Middleware es una función que procesa peticiones:

```
// Estructura: (req, res, next) => void
app.use((req, res, next) => {
  console.log(`${req.method} ${req.path}`);
  next(); // Continuar al siguiente middleware
});

// Flujo de petición:
req → Middleware 1 → Middleware 2 → Route → Respuesta
```

Orden importa: se ejecutan en orden de registro.

4.2 Routing Modular

Mejor práctica: separar rutas en módulos

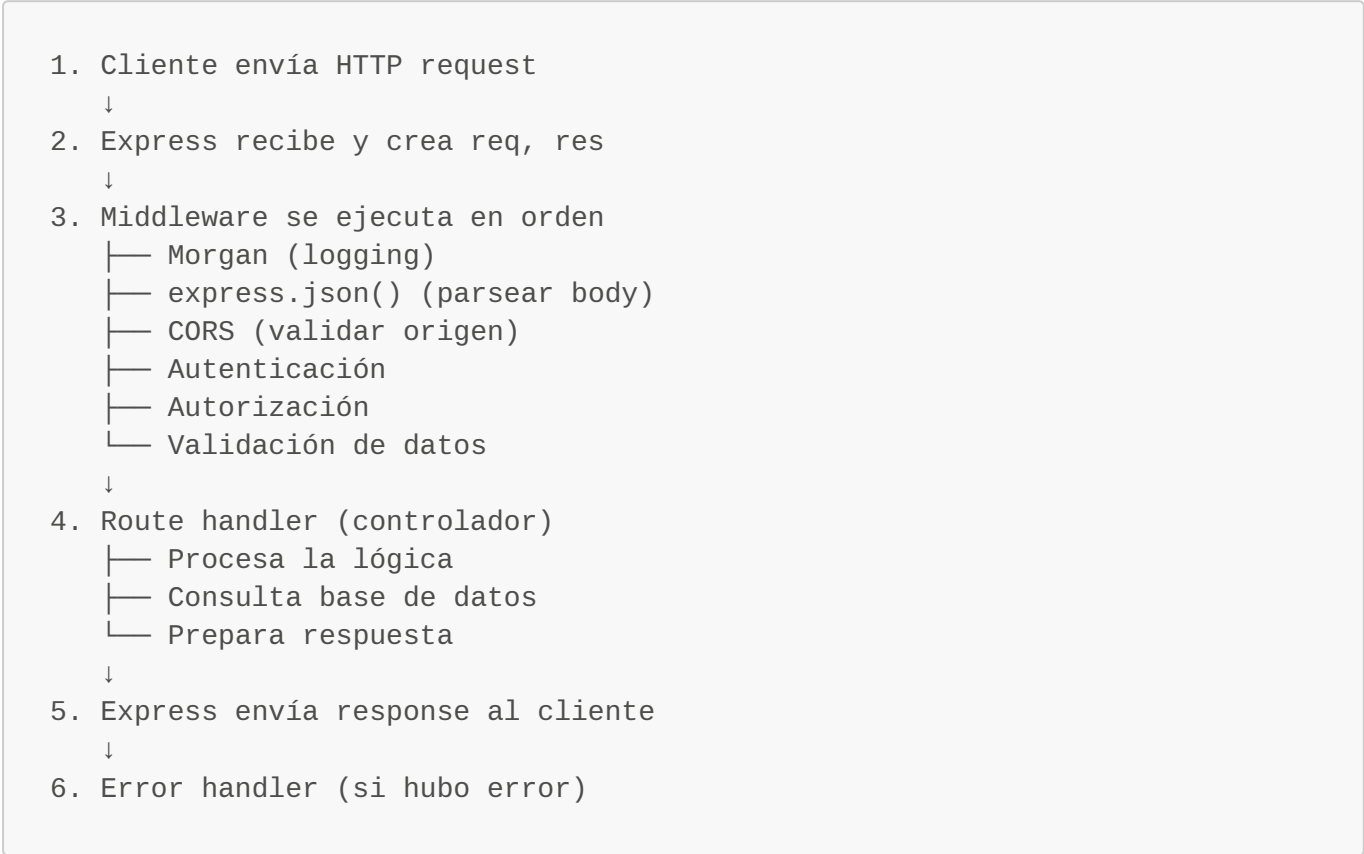
```
// routes/users.routes.js
import { Router } from 'express';
import { getUsers, createUser } from '../controllers/users.controller.js';

const router = Router();
```

```
router.get('/', getUsers);
router.post('/', createUser);
export default router;

// app.js
app.use('/api/users', usersRouter);
```

4.3 Ciclo de Vida de una Petición



5. Bases de Datos: Relacionales vs NoSQL

5.1 Comparación Conceptual

Aspecto	Relacional (SQL)	NoSQL (Documentos)
Esquema	Rígido, predefinido	Flexible, schemaless
Integridad	ACID garantizadas	Eventual consistency
Relaciones	Foreign keys nativas	Referencias manuales
Escalabilidad	Vertical (main)	Horizontal (sharding)
Consultas complejas	JOINS optimizados	Aggregation Pipeline
Ejemplo	PostgreSQL, MySQL	MongoDB, CouchDB

5.2 MongoDB/Mongoose (NoSQL)

Concepto: documentos JSON almacenados en colecciones

```
// Modelo (esquema)
const userSchema = new Schema({
  name: { type: String, required: true },
  email: { type: String, unique: true },
  posts: [{ type: Schema.Types.ObjectId, ref: 'Post' }]
});

// Uso
const user = await User.findById(id).populate('posts');

// Estructura flexible: documentos pueden tener campos diferentes
```

Validación en Mongoose:

```
const schema = new Schema({
  age: {
    type: Number,
    required: true,
    min: 0,
    max: 150,
    validate: {
      validator: v => v % 1 === 0,
      message: 'Debe ser un número entero'
    }
  }
});
```

5.3 PostgreSQL/Prisma (SQL)

Concepto: tablas con filas y columnas, relaciones definidas

```
// Prisma Schema DSL
model User {
  id      Int      @id @default(autoincrement())
  email   String   @unique
  name    String
  posts   Post[]   // Relación 1:N
}

model Post {
  id      Int
  author   User    @relation(fields: [authorId], references: [id])
  authorId Int
}

// Uso: Prisma genera tipos automáticamente
```



```
const user: User = await prisma.user.findUnique({
  where: { id: 1 },
  include: { posts: true }
});
```

Ventajas SQL:

- Transacciones ACID
- JOINS optimizados
- Relaciones explícitas
- Mejor para datos altamente relacionados

Ventajas NoSQL:

- Esquema flexible
- Escalabilidad horizontal
- Rápida prototipación
- Mejor para datos no-relacionados

6. Validación y Manejo de Errores

6.1 Validación con Zod

Zod es una librería de validación type-safe:

```
import { z } from 'zod';

const createUserSchema = z.object({
  name: z.string().min(2).max(100),
  email: z.string().email(),
  password: z.string().min(8),
  age: z.number().int().min(18).optional()
});

// Parsing (lanza error si falla)
const validData = createUserSchema.parse(req.body);

// Validación segura
const result = createUserSchema.safeParse(req.body);
if (!result.success) {
  console.error(result.error.flatten());
}
```

Características avanzadas:

```
// Validación condicional
const schema = z.object({
  type: z.enum(['person', 'company']),
```

```

    name: z.string(),
    taxId: z.string().optional()
  }).refine(
    data => data.type === 'company' ? !!data.taxId : true,
    { message: 'Las empresas requieren Tax ID' }
  );

// Transformación
const schema = z.object({
  email: z.string().email().transform(e => e.toLowerCase())
});

```

6.2 Manejo de Errores

Estrategia en capas:

```

// 1. Validación en entrada
if (!req.body.email) {
  return res.status(400).json({ error: 'Email requerido' });
}

// 2. Lógica de negocio
try {
  const user = await User.findById(id);
  if (!user) throw new AppError('Usuario no encontrado', 404);
} catch (error) {
  // 3. Manejo centralizado
  next(error);
}

// 4. Error middleware global
app.use((err, req, res, next) => {
  const status = err.status || 500;
  const message = err.message || 'Error desconocido';

  res.status(status).json({ error: true, message });
});

```

Patrón AppError:

```

class AppError extends Error {
  constructor(message, status) {
    super(message);
    this.status = status;
  }

  static badRequest(msg) { return new AppError(msg, 400); }
  static unauthorized(msg) { return new AppError(msg, 401); }
  static forbidden(msg) { return new AppError(msg, 403); }
}

```

```
static notFound(msg) { return new AppError(msg, 404); }
```

7. Autenticación y Autorización

7.1 Conceptos Fundamentales

Autenticación ≠ Autorización

- **Autenticación:** verificar identidad (¿Quién eres?)
- **Autorización:** verificar permisos (¿Qué puedes hacer?)

```
// Autenticación: confirmar credenciales
const user = await User.findOne({ email });
if (!user || !await bcrypt.compare(password, user.password)) {
  throw new AppError('Credenciales inválidas', 401);
}

// Autorización: verificar rol/permisos
if (user.role !== 'admin') {
  throw new AppError('Solo admins', 403);
}
```

7.2 JWT (JSON Web Tokens)

JWT es un estándar para tokens stateless:

Header.Payload.Signature

Header	Payload	Signature
<pre>{ alg: HS256 typ: JWT }</pre>	<pre>{ _id: 123, role: ... iat: ... exp: ... }</pre>	<pre>HMACSHA256(header + payload, secret)</pre>
BASE64URL encoded	BASE64URL encoded	BASE64URL encoded

Flujo típico:

```
1. Cliente envía credenciales
↓
```

```
2. Servidor verifica y genera JWT
   ↳ token = sign({ _id, role }, secret, { expiresIn: '7d' })
   ↓
3. Cliente almacena token (localStorage)
   ↓
4. Cliente envía token en headers
   ↳ Authorization: Bearer eyJhbGc...
   ↓
5. Servidor verifica token
   ↳ verify(token, secret) → payload
   ↓
6. Si válido: continúa con permiso
   Si inválido: rechaza (401)
```

7.3 Refresh Tokens

Para mayor seguridad, usar tokens de corta duración:

```
// Access Token: 15 minutos
const accessToken = jwt.sign(payload, secret, { expiresIn: '15m' });

// Refresh Token: 7 días (almacenado en BD)
const refreshToken = jwt.sign(payload, refreshSecret, { expiresIn: '7d' });

// Cuando access token expira
// Cliente envía refresh token
// Servidor genera nuevo access token
```

Ventajas:

- Si access token se expone, exposición limitada
- Refresh token permite logout (eliminar de BD)
- Mayor control del servidor

8. Documentación y Testing

8.1 Swagger/OpenAPI

Swagger documenta automáticamente tu API:

```
// JSDoc comments en rutas
/**
 * @openapi
 * /api/users:
 *   post:
 *     summary: Crear usuario
 *     requestBody:
 *       content:
```

```
*      application/json:
*      schema:
*        type: object
*        properties:
*          email: { type: string }
*      responses:
*        201: { description: Usuario creado }
*        400: { description: Error validación }
*/
app.post('/api/users', createUser);

// Genera interfaz interactiva en /api-docs
```

Beneficios:

- Cliente ve contrato API
- Pueden probar endpoints en navegador
- Se auto-actualiza con cambios
- Documentación viva

8.2 Testing: Jest + Supertest

```
// Test de integración
describe('Auth', () => {
  it('debe registrar usuario', async () => {
    const res = await request(app)
      .post('/api/auth/register')
      .send({ email: 'test@example.com', password: 'pass123' });

    expect(res.status).toBe(201);
    expect(res.body.token).toBeDefined();
  });

  it('debe rechazar email duplicado', async () => {
    await request(app)
      .post('/api/auth/register')
      .send({ email: 'test@example.com', password: 'pass123' });

    const res = await request(app)
      .post('/api/auth/register')
      .send({ email: 'test@example.com', password: 'pass456' });

    expect(res.status).toBe(409);
  });
});
```

Tipo de tests:

- **Unitarios:** función aislada
- **Integración:** múltiples componentes (rutas + BD)

- **End-to-end:** flujo completo desde cliente
-

9. Bases de Datos SQL Modernas

9.1 Supabase: PostgreSQL en la Nube

Supabase proporciona:

- PostgreSQL completo
- Autenticación integrada
- Storage de archivos
- Realtime subscriptions
- Edge Functions

```
// Supabase = PostgreSQL + extras
// Usar con Prisma como ORM
const user = await prisma.user.findUnique({
  where: { email: 'user@example.com' }
});
```

9.2 Migraciones de Base de Datos

Las migraciones versionan cambios en el esquema:

```
# Cambiar schema.prisma
# Generar migración
npx prisma migrate dev --name add_users

# Resultado:
# prisma/migrations/20250115120000_add_users/
# migration.sql ← Cambios en SQL
```

Concepto: BD = código versionado

- Cada cambio es reversible
 - CI/CD aplica automáticamente
 - Equipo siempre sincronizado
-

10. WebSockets: Tiempo Real

10.1 HTTP vs WebSocket

```
HTTP (Request-Response):
Client: "¿Hay mensajes?"
Server: "No"
```

```
Client: "¿Hay mensajes?" (1 segundo después)
Server: "Sí, uno"

WebSocket (Full-Duplex):
Client: (conectado)
Server: (detecta cambio) "¡Nuevo mensaje!"
Client: (recibe al instante)
```

Diferencias clave:

- HTTP: nueva conexión cada petición (overhead ~800 bytes)
- WebSocket: conexión persistente (overhead ~2 bytes)
- HTTP: unidireccional
- WebSocket: bidireccional

10.2 Socket.IO: Abstracción de WebSocket

```
// Servidor
io.on('connection', socket => {
  socket.on('message', data => {
    // Broadcast a todos
    io.emit('message', data);
  });
});

// Cliente
socket.on('message', data => {
  console.log('Recibido:', data);
});
socket.emit('message', 'Hola');
```

Características Socket.IO:

- Reconexión automática
- Fallback a HTTP si no soporta WebSocket
- Rooms (salas para agrupar)
- Namespaces (endpoints separados)

10.3 Casos de Uso

✅ WebSocket ideal para:

- Chat en tiempo real
- Juegos multijugador
- Dashboards en vivo
- Colaboración (Google Docs)

❌ HTTP REST mejor para:

- APIs CRUD
- Cacheo HTTP
- SEO
- Stateless

11. Infraestructura y Deployment

11.1 Ciclo de Vida: Dev → Prod

```
DESARROLLO
↓
npm run dev
├─ Hot reload (watch)
├─ Logs verbose
├─ NODE_ENV=development
└─ BD local
↓
TESTING
↓
npm test
├─ Jest ejecuta tests
├─ Coverage report
└─ CI/CD en GitHub Actions
↓
PRODUCCIÓN
↓
npm run build && npm start
├─ NODE_ENV=production
├─ Logs minimales
├─ BD remota (Supabase/MongoDB Atlas)
├─ HTTPS obligatorio
├─ Docker container
└─ PM2/Kubernetes para escalado
```

11.2 Docker: Containerización

```
# Dockerfile
FROM node:22-alpine

WORKDIR /app
COPY package*.json ./

# Producción: solo dependencias necesarias
RUN npm ci --only=production

COPY . .

# Usuario no-root por seguridad
```



```
USER node
```

```
CMD ["node", "src/app.js"]
```

Ventajas:

- Entorno idéntico en dev/prod
- Fácil escalabilidad
- Aislamiento de dependencias
- Compatible con Kubernetes

11.3 CI/CD: Automatización

```
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm test
      - run: npm run lint

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      # Deploy a Railway/Render/Fly.io
```

Flujo:

```
Push a main
↓
Tests automáticos
├─ Lint
├─ Unit tests
└─ Integration tests
↓
Si pasan → Deploy automático a producción
Si fallan → Notificación, no deploy
```

11.4 Variables de Entorno

```
# .env.example (commitear)
NODE_ENV=development
PORT=3000
DATABASE_URL=mongodb://localhost:27017/myapp
JWT_SECRET=cambia_esto_en_produccion_min_32_chars

# .env (no commitear)
# (Solo en máquina local/servidor)
```

Validar al inicio:

```
const envSchema = z.object({
  NODE_ENV: z.enum(['development', 'production']),
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32)
});

const env = envSchema.parse(process.env);
// Falla rápido si falta algo
```

12. TypeScript: Tipado Estático

12.1 ¿Por Qué TypeScript?

```
JavaScript:
function calculateTotal(items) {
  return items.reduce((sum, item) => sum + item.price, 0);
}
calculateTotal("not an array") // Error en RUNTIME ⚠️

TypeScript:
interface Item { price: number }
function calculateTotal(items: Item[]): number {
  return items.reduce((sum, item) => sum + item.price, 0);
}
calculateTotal("not an array") // Error en COMPILACIÓN ✅
```

Beneficios:

- Errores detectados antes de deploy
- Autocompletado mejorado
- Refactoring seguro
- Documentación viva

12.2 Tipado de Express

```
import { Request, Response, NextFunction } from 'express';

interface CreateUserBody {
  name: string;
  email: string;
  password: string;
}

export const createUser = async (
  req: Request<{}, {}, CreateUserBody>,
  res: Response,
  next: NextFunction
): Promise<void> => {
  try {
    const { name, email, password } = req.body; // Tipado!
    // ...
    res.status(201).json({ user });
  } catch (error) {
    next(error);
  }
};
```

12.3 Genéricos: Reutilizar Tipos

```
// Tipo genérico para respuestas API
interface ApiResponse<T> {
  success: boolean;
  data: T;
  timestamp: Date;
}

type UserResponse = ApiResponse<User>;
type UsersResponse = ApiResponse<User[]>;

// Repository genérico
class Repository<T extends { id: number }> {
  async findById(id: number): Promise<T | null> {
    // ...
  }
}

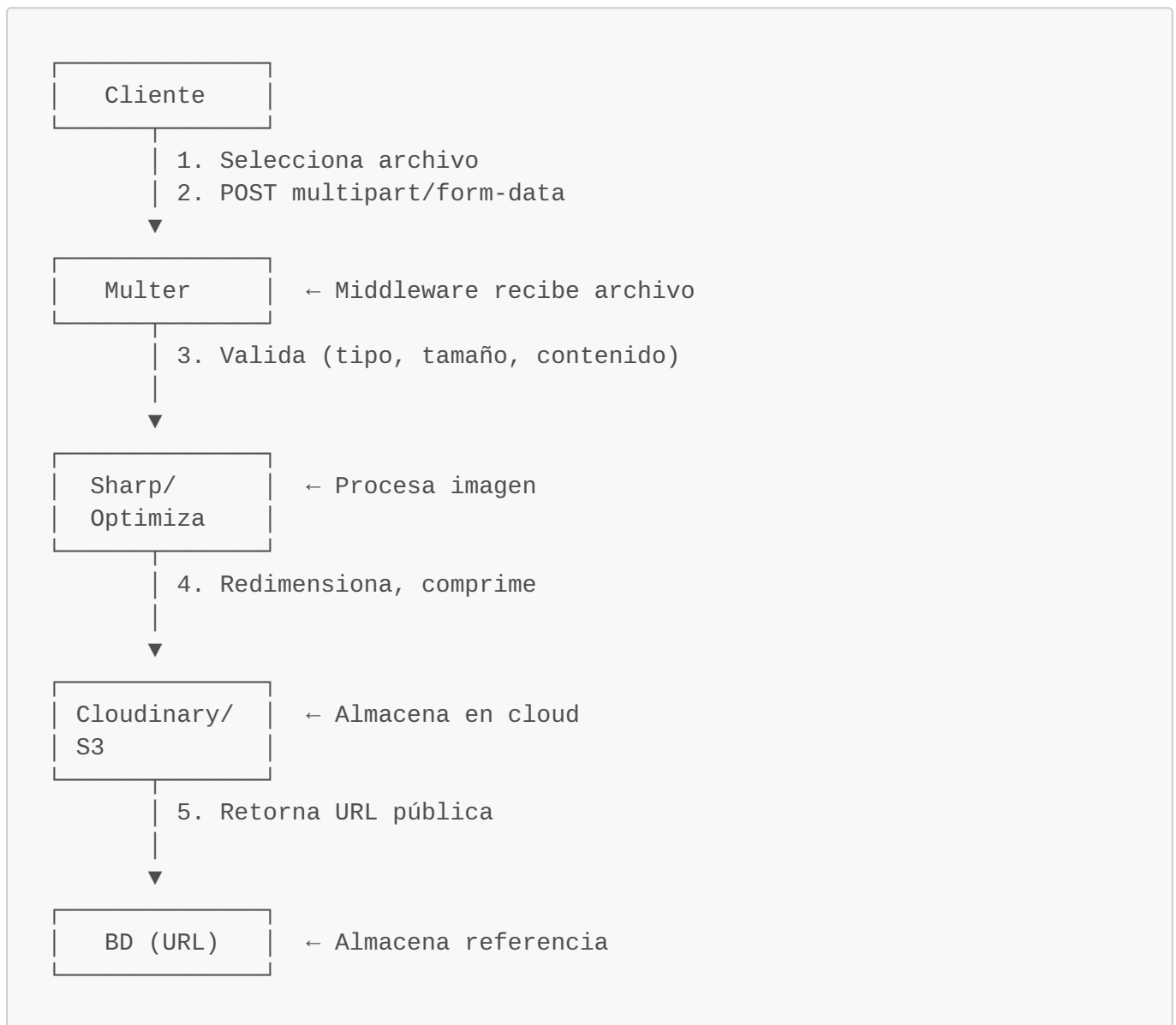
const userRepo = new Repository<User>();
const user = await userRepo.findById(1); // T = User automáticamente
```

13. Storage y Archivos

13.1 Opciones de Almacenamiento

Solución	Ventajas	Contras
Local (filesystem)	Simple, gratuito	No escala, se pierde en redeploy
Cloudinary	CDN integrado, transformaciones	Costos por transformación
Cloudflare R2	Sin costos de egreso	Sin transformaciones nativas
AWS S3	Estándar industria, escalable	Costos de egreso
Supabase Storage	Integrado con BD	Atado a ecosistema

13.2 Flujo de Carga de Archivos



13.3 Seguridad en Uploads

```
// 1. Validar tipo MIME real (no solo extensión)
const type = await fileType.fromBuffer(buffer);
if (!ALLOWED_TYPES.includes(type.mime)) throw new Error('Invalid');

// 2. Limitar tamaño
```

```
if (buffer.length > MAX_SIZE) throw new Error('Too large');

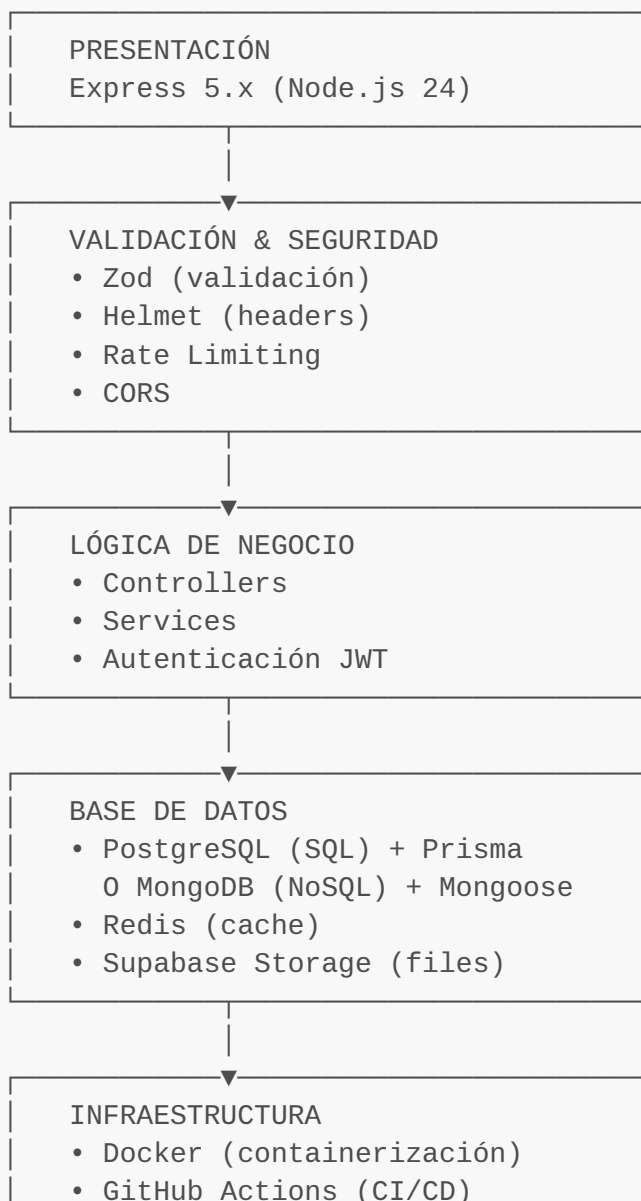
// 3. Escanear contenido malicioso
const suspiciousPatterns = [/<script/i, /onclick/i];
if (suspiciousPatterns.some(p => p.test(content))) throw new
Error('Malicious');

// 4. Generar nombre único
const filename = crypto.randomBytes(16).toString('hex') + ext;

// 5. Rate limiting por usuario
const uploads = userUploads.get(userId) || 0;
if (uploads >= LIMIT) throw new Error('Rate limited');
```

14. Arquitectura Completa: De Cero a Producción

14.1 Stack Recomendado 2025



- Railway/Render/Fly.io (deploy)
- PM2 (process management)
- Monitoring (Sentry, UptimeRobot)

14.2 Flujo Completo de una Petición Autenticada

```
// 1. ENTRADA
GET /api/users/123
Headers: Authorization: Bearer eyJhbGc...

// 2. MIDDLEWARE DE SEGURIDAD
├ CORS: ¿origen permitido?
├ Helmet: agregar headers seguridad
├ Rate Limit: ¿usuario ha excedido límite?
└ Auth: ¿token válido?

// 3. ROUTER
└ GET /users/:id → usersController.getUser

// 4. CONTROLADOR
usersController.getUser():
├ Extraer id de req.params
├ Validar con Zod (si requerido)
├ Verificar autorización
│ (¿cliente puede ver este usuario?)
└ Llamar a servicio

// 5. SERVICIO
userService.getUserById(id):
├ Consultar BD
├ Si cache hit, retornar
├ Transformar respuesta
└ Guardar en cache

// 6. SALIDA
HTTP 200 OK
{
  success: true,
  data: {
    id: 123,
    name: 'Ana García',
    email: 'ana@example.com'
  },
  timestamp: "2025-01-15T10:30:00Z"
}
```

14.3 Manejo de Errores en Cascada

```
// Cualquier error en la cadena es capturado

const handler = async (req, res, next) => {
  try {
    // Validación
    if (!req.body.email) {
      return res.status(400).json({ error: 'Email required' }); // Early return
    }

    // Lógica
    const user = await userService.getUser(req.params.id);
    if (!user) {
      throw new AppError('User not found', 404); // Error manejado
    }

    // Éxito
    res.json({ data: user });
  } catch (error) {
    next(error); // Pasa al error middleware
  }
};

// Error middleware global
app.use((err, req, res, next) => {
  const status = err.status || 500;
  const message = err.message || 'Server error';

  if (process.env.NODE_ENV === 'development') {
    res.status(status).json({ error: true, message, stack: err.stack });
  } else {
    res.status(status).json({ error: true, message });
  }
});
```

15. Patrones de Diseño

15.1 MVC (Model-View-Controller)

```
User Request
  ↓
Route
  ↓
Controller → Recibe request, orquesta
  ↓
Service → Lógica de negocio
  ↓
Model → Consulta BD
```

↓
View (JSON Response)

Ventajas:

- Separación de responsabilidades
- Testeable
- Mantenible

15.2 Repository Pattern

```
// Abstrae acceso a datos
interface IUserRepository {
  findById(id: number): Promise<User | null>;
  findAll(): Promise<User[]>;
  create(data: CreateUserDTO): Promise<User>;
  update(id: number, data: UpdateUserDTO): Promise<User>;
  delete(id: number): Promise<void>;
}

// Implementación intercambiable
class PrismaUserRepository implements IUserRepository {
  async findById(id: number) {
    return prisma.user.findUnique({ where: { id } });
  }
  // ...
}

// Usar en servicio
class UserService {
  constructor(private repo: IUserRepository) {}

  async getUser(id: number) {
    return this.repo.findById(id);
  }
}
```

15.3 Dependency Injection

```
// Sin DI (acoplado)
class UserService {
  private repo = new UserRepository(); // Crea internamente
}

// Con DI (desacoplado)
class UserService {
  constructor(private repo: IUserRepository) {} // Inyectado
}
```



```
// Uso
const repo = new UserRepository();
const service = new UserService(repo);

// Fácil testear con mock
const mockRepo = { findById: jest.fn() };
const service = new UserService(mockRepo);
```

16. Buenas Prácticas

16.1 Código Limpio

```
// ❌ Malo: vago, sin contexto
const proc = async (d) => {
  const r = await db.query('SELECT * FROM users');
  return r.map(x => ({ ...x, t: Date.now() }));
};

// ✅ Bueno: claro, autodocumentado
async function getAllUsersWithTimestamp() {
  const users = await database.getAllUsers();
  return users.map(user => ({
    ...user,
    timestamp: Date.now()
  }));
}
```

16.2 Manejo de Concurrency

```
// ❌ Secuencial (lento)
const user = await User.findById(1);
const posts = await Post.find({ userId: 1 });
const comments = await Comment.find({ userId: 1 });

// ✅ Paralelo (rápido)
const [user, posts, comments] = await Promise.all([
  User.findById(1),
  Post.find({ userId: 1 }),
  Comment.find({ userId: 1 })
]);
```

16.3 Logging Estructurado

```
// ❌ Malo: strings no parseables
console.log('User login attempt from 192.168.1.1 at 10:30');
```

```
//  Bueno: JSON estructurado
logger.info({
  event: 'user:login',
  userId: 123,
  ip: '192.168.1.1',
  timestamp: new Date().toISOString(),
  success: true
});

// Fácil de parsear y filtrar en logs centralizados
```

16.4 Graceful Shutdown

```
const server = app.listen(3000);

process.on('SIGTERM', async () => {
  console.log('Recibido SIGTERM, cerrando...');

  server.close(async () => {
    // Cerrar conexiones
    await mongoose.connection.close();
    await redis.quit();

    console.log('Servidor cerrado correctamente');
    process.exit(0);
  });

  // Forzar cierre si tarda mucho
  setTimeout(() => process.exit(1), 10000);
});
```

17. Resumen: Conceptos Clave por Nivel

Nivel 1: Fundamentos

-  HTTP request/response
-  Express routing
-  Async/await
-  CRUD básico

Nivel 2: Intermedio

-  Validación con Zod
-  Autenticación JWT
-  Mongoose/Prisma
-  Error handling
-  Testing con Jest

Nivel 3: Avanzado

- ☒ WebSockets tiempo real
- ☒ Transacciones BD
- ☒ Optimizaciones (caching, indexing)
- ☒ Arquitectura microservicios
- ☒ CI/CD, Docker, Kubernetes

Nivel 4: Producción

- ☒ Monitoring y alertas
- ☒ Rate limiting
- ☒ DDoS protection
- ☒ Multi-región deployment
- ☒ Performance tuning

18. Hoja de Ruta de Aprendizaje

Semana 1-2: Fundamentos

- └ Node.js, ESM, Event Loop
- └ HTTP, REST, Express
- └ Routing básico

Semana 3-4: BD y Datos

- └ MongoDB + Mongoose
- └ Validación con Zod

Semana 5-6: Autenticación

- └ JWT, bcrypt
- └ Middlewares

Semana 7-8: Calidad

- └ Testing con Jest
- └ Documentación Swagger

Semana 9-10: Bases de Datos SQL

- └ PostgreSQL
- └ Prisma ORM

Semana 11-12: Tiempo Real

- └ WebSockets
- └ Socket.IO

Semana 13-14: DevOps

- └ Docker
- └ CI/CD
- └ Deploy

Semana 15-16: Avanzado

- └ TypeScript

- └ Optimizaciones
- └ Patrones arquitectónicos

19. Referencias y Recursos

Documentación Oficial:

- [Node.js](#)
- [Express](#)
- [MongoDB/Mongoose](#)
- [PostgreSQL/Prisma](#)
- [Socket.IO](#)
- [TypeScript](#)

Librerías Clave:

- [zod](#): Validación
- [jsonwebtoken](#): JWT
- [bcryptjs](#): Hashing
- [passport](#): Autenticación
- [jest](#): Testing
- [docker](#): Containerización
- [pm2](#): Process management

Plataformas de Deploy:

- [Railway](#)
- [Render](#)
- [Fly.io](#)
- [Heroku](#)
- [AWS](#)
- [DigitalOcean](#)

Conclusión: Node.js en 2025 es un ecosistema maduro y robusto para backend production. La clave es entender los fundamentos (Event Loop, async), dominar un framework (Express), elegir la BD correcta (SQL/NoSQL), y automatizar el deployment (Docker + CI/CD).